

Dados presentes no artigo já publicado

1. Análise de Eficiência Hídrica e Volumétrica (Potencial de Economia)

Os dados textuais fornecem uma base de consumo bruto por ciclo baseada em especificações de mercado (Ypê e INMETRO) e no próprio estudo de caso das máquinas utilizadas para testes (Brastemp BWL09B e BWK12A).

- Capacidade de Retenção de Água: O projeto estabelece que o descarte de água varia drasticamente conforme o porte da lavadora:
 - *Máquinas de até 10 kg (como o modelo padrão de 11 kg ou similares):* Consumo aproximado de 115 a 135 litros por ciclo completo.
 - *Máquinas de grande porte (modelos de 17 kg):* Consumo de até 197 litros por ciclo completo.
- Regra de Negócio do Software (Filtragem de Fluxos): Como a primeira arquitetura com duas válvulas foi descartada, o sistema passou a ignorar o 1º ciclo (água com alta concentração de sujeira/sabão). O foco do reuso passou a ser estritamente o 2º fluxo (enxágue) e o 3º fluxo (enxágue extra), controlados pelas posições lógicas da interface física:
 - Configuração 1: Intercepta e reaproveita apenas o 2º ciclo.
 - Configuração 2: Intercepta o 2º ciclo + o enxágue extra.

Impacto Quantitativo: Considerando que o enxágue representa uma fração significativa do descarte total, o potencial de economia direta por lavagem residencial varia de 115 a 230 litros de água potável por utilização semanal, dependendo do tamanho da máquina e do número de ciclos interceptados.

2. Análise de Engenharia e Confiabilidade (Evolução Iterativa)

O documento detalha uma abordagem experimental iterativa. Analisando qualitativamente os estados do sistema, identificamos os gargalos técnicos superados:

[Arquitetura 1] —> Falha de Pressão (Gotejamento) —> Exigiria Bomba Mecânica (Inviável) [cite: 105, 106, 107]

|
[Arquitetura 2] —> Transição para 1 Eletroválvula (EVS) + Galão de 5L por Gravidade [cite: 109, 114]

|
[Protótipo Final] —> Correção de Ruídos: Soldas reforçadas + Software com Lógica de Debounce [cite: 116, 117, 141]

- Ponto Crítico de Hardware: A primeira versão falhou porque a válvula selecionada não operava adequadamente sem pressão hidrostática (gerando apenas gotejamento). A mudança para uma junção em Y associada a uma única EVS eliminou a necessidade de uma bomba elétrica, preservando o princípio de baixo custo e eficiência energética.

- Instabilidade de Sinal (Tratamento de Dados): O movimento irregular e a turbulência da água cinza ao entrar no galão de 5 litros geravam oscilações mecânicas na boia flutuante (*reed switch*). Isso provocava leituras falsas (ruído de sinal), corrigidas no software através da implementação de uma lógica de *debounce*, garantindo leituras lógicas estáveis antes de alterar o estado do relé.

3. Análise Financeira e de Viabilidade Económica (ROI)

O texto apresenta um comparativo de retorno financeiro baseado em estudos de caso nacionais e internacionais para validar o custo-benefício do protótipo baseado em Arduino Pro Mini:

- Ponto de Inflexão (Payback): Mesmo considerando perdas operacionais mínimas no descarte de segurança, o tempo de retorno do investimento estimado para o dispositivo em produção artesanal ficou estabelecido entre 1,9 e 2,3 anos.
- Comparativo de Mercado: Esse tempo é altamente competitivo se comparado a sistemas internacionais robustos de reuso residencial (como o estudo citado no Canadá, cujo retorno variava de 11 a 52 anos devido ao alto custo de instalação).
- Ganho de Escala: O relatório infere que a produção industrial em larga escala reduziria drasticamente o custo marginal dos componentes (microcontrolador, relé e EVS), encurtando o *payback* para menos de um ano e viabilizando a adoção por famílias de baixa renda.

Dados pesquisados para a atividade e complementação da fundamentação teórica

1. Introdução e Contextualização

O projeto de automação para reuso de águas cinzas residenciais (focado em máquinas de lavar de 9 kg a 17 kg, com descarte de 115L a 197L por ciclo) ganha relevância quando contrastado com a realidade macroeconômica do país.

A partir de um pipeline de ETL aplicado a dados de **5.298 municípios brasileiros**, constatou-se que a malha industrial nacional possui uma **vazão captada crítica de 192.412,56 L/s**. Deste total, **54,53% (104.915,43 L/s) é consumido de forma definitiva** (perdas/processos), retornando apenas 87.497,09 L/s aos mananciais.

Grandes centros urbanos lideram esse estresse hídrico, com destaque para **São Paulo (5.199,38 L/s)** e **Rio de Janeiro (5.013,69 L/s)**. Estes dados fundamentam a urgência de soluções descentralizadas de reuso doméstico (como o dispositivo desenvolvido): ao poupar a água potável na ponta consumidora, reduz-se a pressão de captação sobre as bacias hidrográficas sobrecarregadas dessas regiões de alta densidade.

2. Descrição Técnica do Pipeline de ETL

O algoritmo foi estruturado em Python (biblioteca Pandas) seguindo três etapas automatizadas:

- **Extração e Padronização:** Leitura da base bruta e mapeamento de colunas com nomenclaturas técnicas codificadas (QRET_LS_15, QCON_LS_15, QREN_LS_15)

para variáveis intuitivas (vazao_captada_l_s, vazao_consumida_l_s, vazao_retornada_l_s). Os nomes dos municípios foram limpos de espaços e padronizados em caixa alta.

- **Limpeza e Engenharia de Atributos:** Valores nulos foram convertidos em 0 para não corromper o cálculo estatístico. Criou-se a métrica taxa_consumo_efetivo_porcentagem através de uma estrutura lógica (np.where) que calcula a eficiência local blindando o sistema contra erros de divisão por zero.
- **Carga:** Exportação de um novo dataset estruturado (dataset_uso_agua_industria_limpo.csv) pronto para consumo analítico.

Código Python:

```
import os
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Configuração visual dos gráficos
sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)

# 1. EXTRAÇÃO E PADRONIZAÇÃO (Extract & Format)
try:
    df_raw = pd.read_csv('dados.csv', encoding='utf-8')
    print("Base de dados carregada com sucesso!")
except FileNotFoundError:
    print("Erro: O arquivo CSV não foi encontrado na pasta atual.")
    exit()

# Dicionário para renomear colunas técnicas para nomes intuitivos
colunas_novas = {
    'NOME': 'municipio',
    'ID_MUNICIP': 'id_municipio',
    'QRET_POR': 'percentual_vazao_nacional',
    'QRET_LS_15': 'vazao_captada_l_s', # Litros por segundo retirados
    'QCON_LS_15': 'vazao_consumida_l_s', # Litros por segundo
    'QREN_LS_15': 'vazao_retornada_l_s' # Litros por segundo que
    efetivamente consumidos
    voltam ao meio ambiente
}

df = df_raw.rename(columns=colunas_novas)[list(colunas_novas.values())]

# Limpeza básica
```

```

df = df.dropna(subset=['municipio'])
df['municipio'] = df['municipio'].str.strip().str.upper()
df = df.drop_duplicates()

# Preencher nulos numéricos com zero
colunas_num = ['vazao_captada_l_s', 'vazao_consumida_l_s',
               'vazao_retornada_l_s', 'percentual_vazao_nacional']
df[colunas_num] = df[colunas_num].fillna(0)

# 2. ENGENHARIA DE DADOS (Dataset & Novas Métricas)
# Criando métrica de eficiência: Percentual da água captada que é
# consumida pela indústria
# Usando np.where para evitar divisão por zero
import numpy as np
df['taxa_consumo_efetivo_porcentagem'] = np.where(
    df['vazao_captada_l_s'] > 0,
    (df['vazao_consumida_l_s'] / df['vazao_captada_l_s']) * 100,
    0
)

# Salvar o Dataset estruturado e limpo
df.to_csv('dataset_uso_agua_industria_limpo.csv', index=False,
encoding='utf-8')
print("📁 Dataset estruturado salvo como
'dataset_uso_agua_industria_limpo.csv'")

# 3. GERAÇÃO DE INFORMAÇÕES RELEVANTES (Insights)
print("\n" + "="*50)
print("      RELATÓRIO EXECUTIVO: USO DE ÁGUA NA INDÚSTRIA")
print("="*50)

total_captado = df['vazao_captada_l_s'].sum()
total_consumido = df['vazao_consumida_l_s'].sum()
total_retornado = df['vazao_retornada_l_s'].sum()

print(f"♦ Vazão Total Captada pela Indústria: {total_captado:,.2f}
L/s")
print(f"♦ Vazão Total Consumida (Perda/Processo):
{total_consumido:,.2f} L/s")
print(f"♦ Vazão Total Retornada aos Mananciais: {total_retornado:,.2f}
L/s")

```

```

print(f"💎 Índice Médio de Consumo Efetivo:
{ (total_consumido/total_captado)*100:.2f}% da água captada")

# Identificar os 5 municípios com maior impacto industrial hídrico
print("\n📈 TOP 5 Municípios com Maior Captação Industrial (L/s):")
top_captacao = df.nlargest(5, 'vazao_captada_l_s')[['municipio',
'vazao_captada_l_s', 'vazao_consumida_l_s']]
print(top_captacao.to_string(index=False))

print("\n📈 TOP 5 Municípios com Maior Consumo Líquido Industrial
(L/s):")
top_consumo = df.nlargest(5, 'vazao_consumida_l_s')[['municipio',
'vazao_consumida_l_s', 'vazao_retornada_l_s']]
print(top_consumo.to_string(index=False))

# 4. DATA VISUALIZATION (Geração de Gráficos)
print("\n" + "="*50)
print("                GERANDO GRÁFICOS ANALÍTICOS")
print("="*50)

# Criar pasta para salvar os gráficos se não existir
if not os.path.exists('graficos_analise'):
    os.makedirs('graficos_analise')

# Gráfico 1: Distribuição Geral da Água (Captada vs Consumida vs
Retornada)
plt.figure()
valores_totais = [total_captado, total_consumido, total_retornado]
labels_totais = ['Água Captada (L/s)', 'Água Consumida (L/s)', 'Água
Retornada (L/s)']
sns.barplot(x=labels_totais, y=valores_totais, palette='Blues_r')
plt.title('Balanço Geral do Volume Hidrológico Industrial',
fontsize=14, fontweight='bold')
plt.ylabel('Volume Total (Litros por Segundo)')
plt.savefig('graficos_analise/01_balanco_hidrico_total.png',
bbox_inches='tight')
plt.close()
print("🖼️ Gráfico 1: '01_balanco_hidrico_total.png' gerado.")

# Gráfico 2: Maiores Consumidores (Top 10 Municípios)
plt.figure()
top_10_municipios = df.nlargest(10, 'vazao_captada_l_s')

```

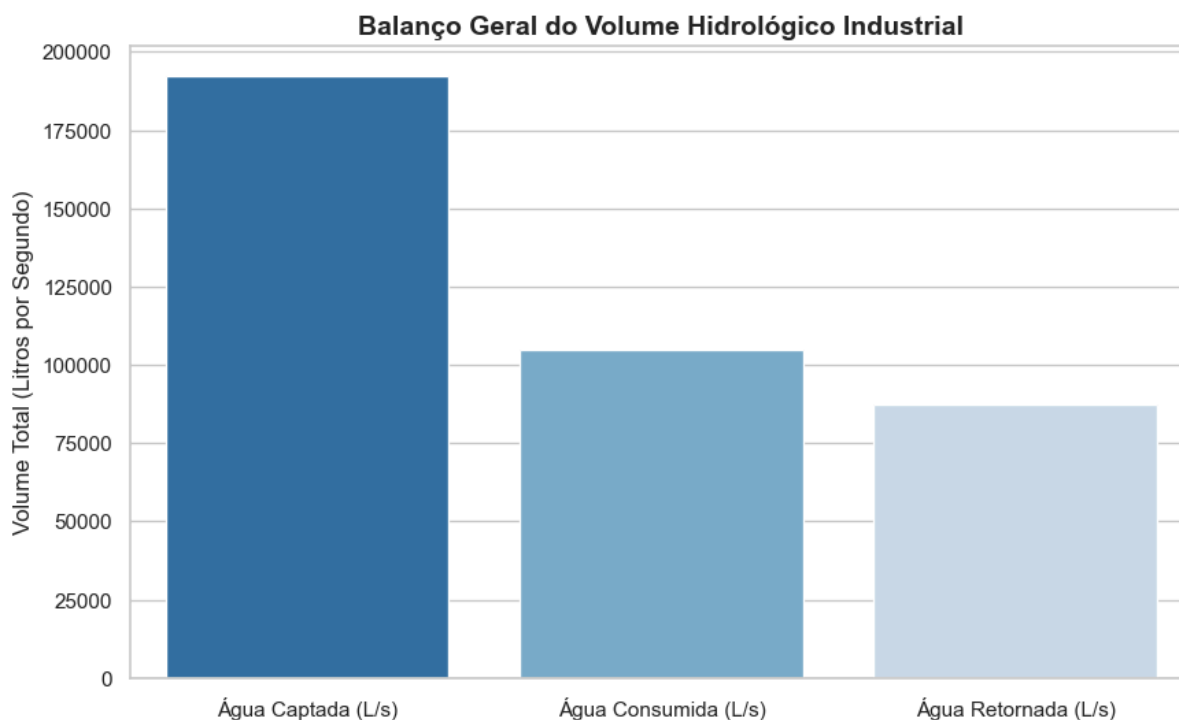
```

sns.barplot(data=top_10_municipios, x='vazao_captada_l_s',
y='municipio', palette='viridis')
plt.title('Top 10 Municípios por Captação de Água Industrial',
fontsize=14, fontweight='bold')
plt.xlabel('Vazão Captada (Litros por Segundo)')
plt.ylabel('Município')
plt.savefig('graficos_analise/02_top_10_municipios_captacao.png',
bbox_inches='tight')
plt.close()
print("📊 Gráfico 2: '02_top_10_municipios_captacao.png' gerado.")

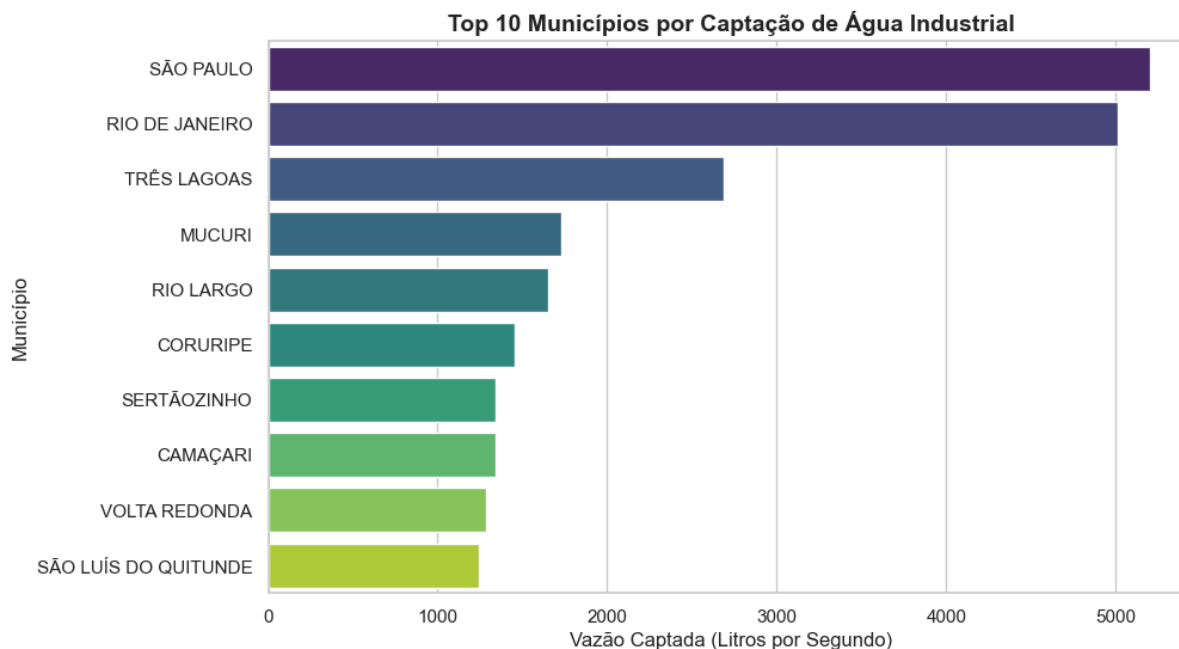
# Gráfico 3: Relação entre Captação e Retorno (Dispersão)
plt.figure()
# Filtrando apenas cidades com captação relevante para evitar poluição
visual no gráfico
df_filtrado_grafico = df[df['vazao_captada_l_s'] > 10]
sns.scatterplot(data=df_filtrado_grafico, x='vazao_captada_l_s',
y='vazao_retornada_l_s', alpha=0.7, color='teal')
plt.title('Correlação: Volume Captado vs Volume Retornado (Cidades > 10
L/s)', fontsize=12, fontweight='bold')
plt.xlabel('Vazão Captada (L/s)')
plt.ylabel('Vazão Retornada (L/s)')
plt.savefig('graficos_analise/03_correlacao_captacao_retorno.png',
bbox_inches='tight')
plt.close()
print("📊 Gráfico 3: '03_correlacao_captacao_retorno.png' gerado.")

print("\n🚀 Concluído! Verifique a pasta 'graficos_analise' para
visualizar os insights gerados.")

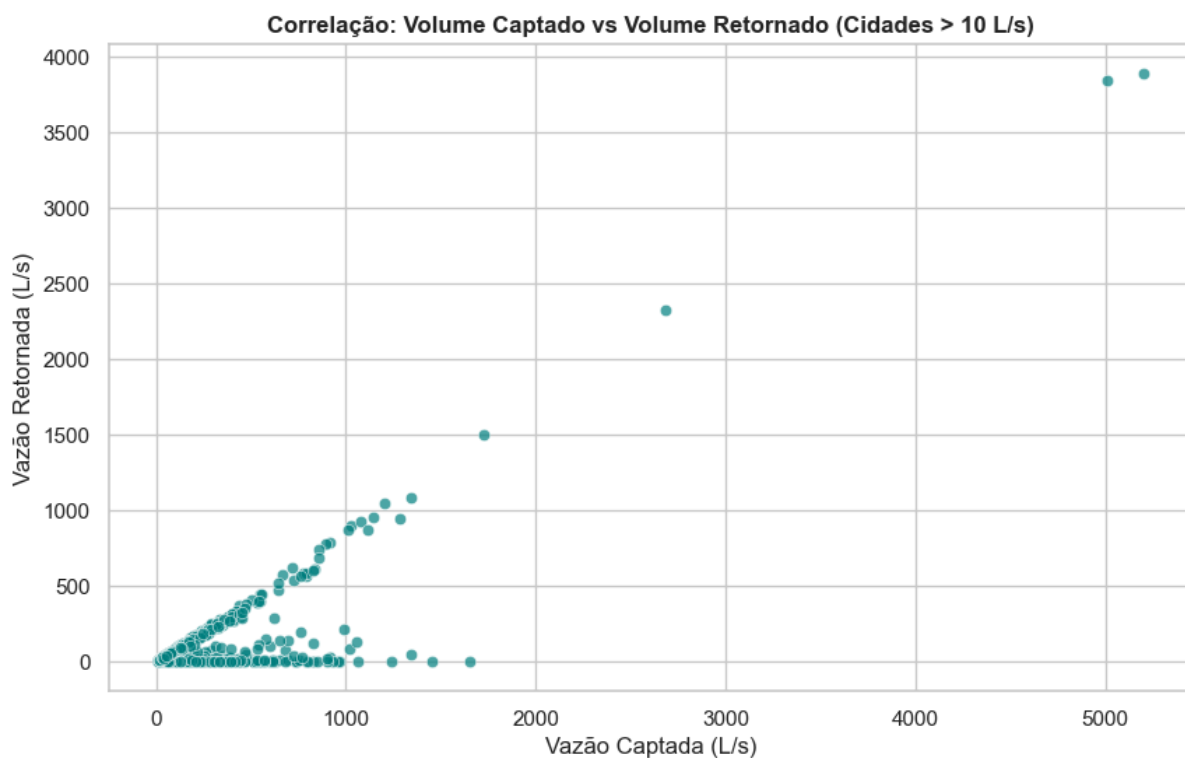
```



- **Gráfico 01 (Balanco Hidrológico Total):** Um gráfico de barras que ilustra o *gap* volumétrico nacional entre captação e retorno, justificando visualmente a necessidade de sistemas de circuito fechado e reuso de efluentes.



- **Gráfico 02 (Top 10 Municípios de Maior Captação):** Um gráfico de classificação (ranking) que comprova a concentração de demanda hídrica em capitais e polos industriais, validando os mercados prioritários para a implementação do ecossistema de reuso.



- **Gráfico 03 (Dispersão Captação vs. Retorno):** Um diagrama de dispersão para cidades com fluxo acima de 10 L/s, evidenciando a heterogeneidade no comportamento de consumo hídrico regional e reforçando a importância de dispositivos físicos adaptáveis e inteligentes (como a lógica de seleção de ciclos do protótipo).